GStar Assignment 1

Approach and key learnings

I worked through the forward-pass problems incrementally, starting with tiled attention in PyTorch and a weighted row-sum kernel in Triton. I then reused the online softmax update for non-causal attention, causal attention, grouped query attention (GQA), sliding windows, and attention sinks. I kept the supplied kernel structure and fixed tile sizes rather than attempting extensive performance tuning.

The central idea was to keep a running maximum, denominator, and weighted-value accumulator for each query row. When a new tile raises the maximum, both previous accumulators must be rescaled before adding the tile's contribution. This avoids storing the full attention matrix. In Triton, each program handles a query tile and streams through the relevant K/V tiles. Since the scores include a `log2(e)` factor, the update consistently uses `exp2`.

For causal attention, I split earlier key tiles from the diagonal region, where each query needs its own causal mask. GQA changes which K/V head is loaded: each consecutive group of query heads shares one K/V head. For sliding windows, I computed the earliest key in token coordinates, rounded down to a tile boundary, and applied the exact window mask per query. Attention sinks add a prefix of keys. I processed sink tiles before the window separately, then used a sink-or-window mask in the remaining phases so overlapping keys were counted once.

Most of the debugging came from details that were easy to miss when translating the math. Concatenating a 64-column score tile with the running maximum created a 65-column Triton tensor; reducing the scores first and using `tl.maximum` fixed that. Forgetting to update the running maximum discarded earlier contributions. I also had to parenthesize Boolean comparisons and distinguish token offsets from block indices. Reloading K/V in another phase made sense once I tracked the actual token positions: the query tile stays fixed, but the keys and values change.

The trickiest numerical case was a row with no allowed keys in an early window tile. Its old and new maxima are both negative infinity, so subtracting them produces NaN. Using a finite reference only for that subtraction makes the empty tile contribute zero while keeping the true running maximum unchanged. The same softmax state then carries across all phases.